

3. naloga: Določanje elastičnih konstant v nematskem tekočem kristalu

1 Uvod

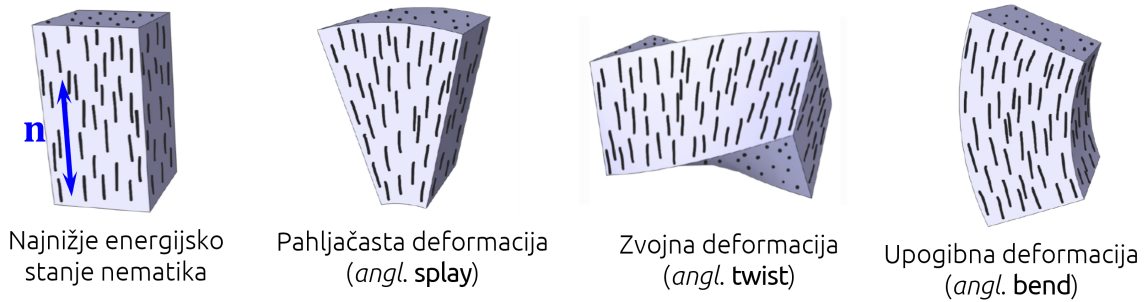
Uporaba tekočih kristalov pogosto temelji na njihovih elastičnih lastnostih. V tej nalogi bomo spoznali potencialen nov način za določitev elastičnih konstant v nematskem tekočem kristalu s pomočjo nevronske mreže iz meritev intenzitete prepuščene svetlobe.

1.1 Nematski tekoči kristali

Orientacijski red v nematskem tekočem kristalu opišemo z direktorskim poljem $\mathbf{n}(\mathbf{r})$, ki v vsaki točki kaže v smeri lokalne ureditve gradnikov. Pri tem se zavedamo, da sta smeri $\mathbf{n}$ in $-\mathbf{n}$ ekvivalentni, kar se mora odražati tudi v nadaljnjem fizikalnem opisu. Deformacije znotraj direktorskega polja opišejo odvodi $\partial_i n_j$, ki nam pomagajo, da skonstruiramo Frank-Oseenovo gostoto elastične proste energije

$$f_{elastic} = \frac{1}{2}K_{11}(\nabla \cdot \mathbf{n})^2 + \frac{1}{2}K_{22}(\mathbf{n} \cdot \nabla \times \mathbf{n})^2 + \frac{1}{2}K_{33}(\mathbf{n} \times \nabla \times \mathbf{n})^2. \quad (1)$$

K_{11} , K_2 in K_{33} so različne deformacijske konstante. Vsak člen predstavlja enega od osnovnih deformacijskih načinov, to so pahljačaste, zvojne ter upogibne deformacije, prikazane na sliki 1.



Slika 1: Vizualizacija osnovnih deformacijskih načinov v nematskem tekočem kristalu – pahljačasti (splay), zvojni (twist) in upogibni (bend).

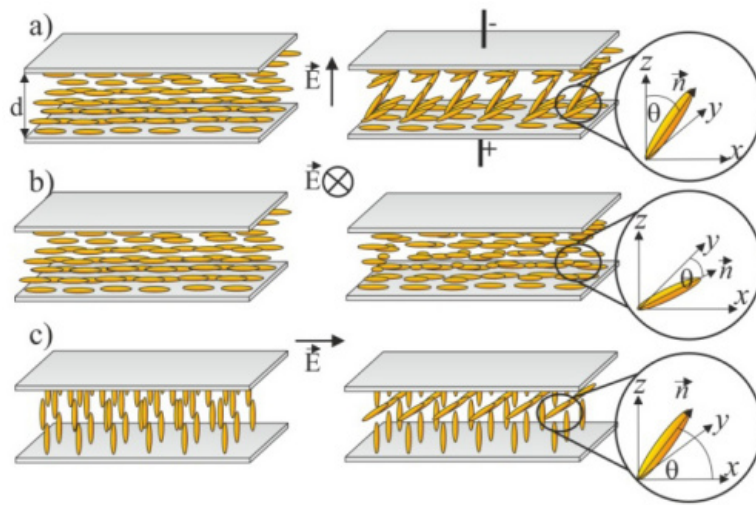
Eksperimentalno določanje deformacijskih konstant izkorišča odziv tekočega kristala na zunanje električno polje $\mathbf{E}$, ki gostoti proste energije doda prispevek

$$f_{electric} = -\frac{1}{2}\epsilon_0\epsilon_a(\mathbf{n} \cdot \mathbf{E})^2, \quad (2)$$

kjer je ϵ_a dielektrična anizotropija tekočega kristala (ekvivalentno bi lahko obravnavali odziv na zunanje magnetno polje). V ta namen tekoči kristal postavimo med dve plošči na razdalji d (debelina celice), in med njima vzpostavimo homogeno električno polje v želeni smeri. Izkazuje se, da majhne jakosti polja ne spremenijo orientacije direktorja – ta nastopi šele, ko je presežena kritična jakost

$$E_c = \frac{\pi}{d} \sqrt{\frac{K_i}{\epsilon_0 \epsilon_a}}, \quad (3)$$

kjer je K_i katerakoli od deformacijskih konstant. Pojav se imenuje Fréederickszov prehod. V ozadju enačbe (3) se skriva predpostavka, da smo z ustrezno izbiro smeri zunanega polja uspeli vzbuditi le enega od osnovnih deformacijskih načinov. To je dosegljivo ob ustrezni izbiri sidranja, ki fiksira orientacijski red na robu celice (slika 2). Najbolj pogosta sta homeotropno sidranje ($\mathbf{n}$ pravokoten na rob celice) ter degenerirano planarno sidranje ($\mathbf{n}$ vzporeden z robom).



Slika 2: Prikaz meritve elastičnih konstant (a) K_{11} , (b) K_{22} in (c) K_{33} preko Fréederickszovega prehoda. V primerih (a) in (b) je uporabljeno planarno sidranje, v primeru (c) pa homeotropno [1].

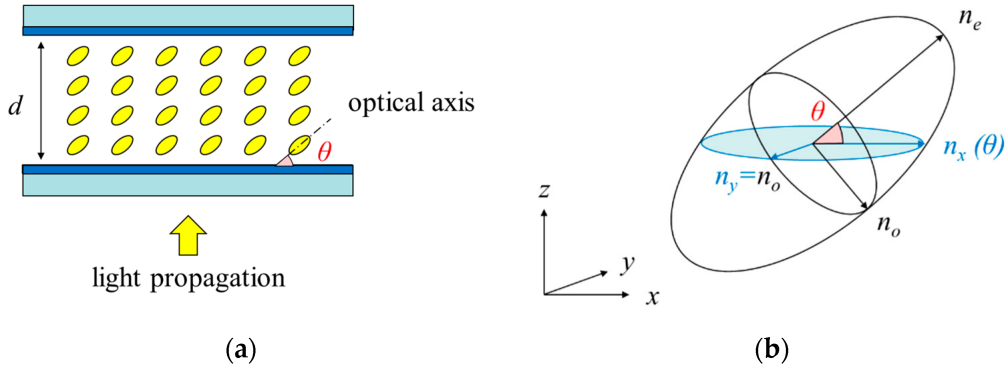
Točka Fréederickszovega prehoda je določena z merjenjem prepuščene svetlobe ali kapacitivnosti. Kljub konceptualni preprostosti ta način določanja elastičnih konstant eksperimentalno ni trivialen, saj je težko natančno kontrolirati eksperimentalne pogoje in rezultati v literaturi močno variirajo tudi za pogosto uporabljene materiale, kot je na primer 5CB. Tako bi metoda, ki se ne zanaša na električni odziv, lahko pripomogla k poenostavitvi meritev elastičnih konstant.

Elastičnost tekočih kristalov se pogosto izkorišča v povezavi z njihovimi optičnimi lastnostmi. Ureditvena anizotropija povzroči dvolomnost, kjer je optična os orientirana v smeri direktorja $\mathbf{n}$. Svetlobni žarek, ki potuje skozi tekoči kristal lahko torej razdelimo na redno in izredno komponento. Propagacija rednega žarka ni odvisna od smeri direktorja in je pogojena le z velikostjo rednega lomnega količnika n_o . Nasprotno je propagacija izrednega žarka odvisna od kota θ med ravnino, pravokotno na valovni vektor $\mathbf{k}$, ter direktorjem $\mathbf{n}$ (glej sliko 3), za lomni količnik izrednega žarka velja

$$\frac{1}{n(\theta)^2} = \frac{\cos^2 \theta}{n_e^2} + \frac{\sin^2 \theta}{n_o^2}, \quad (4)$$

kjer je n_e velikost izrednega lomnega količnika ob propagaciji svetlobe pravokotno na optično os (pri $\theta = 0$). Skica za primer propagacije svetlobe v smeri osi z je prikazana na sliki 3. Orientacija direktorja se lahko seveda po poti žarka spreminja, tako da je $\theta = \theta(z)$ in posledično $n = n(z)$. Razlika v akumulirani fazi, ko oba žarka prepotujeta razdaljo d skozi tekoči kristal je

$$\Gamma = \frac{2\pi}{\lambda} \int_0^d (n(z) - n_o) dz. \quad (5)$$



Slika 3: Propagacija svetlobe skozi nematski tekoči kristal. Valovni vektor $\mathbf{k}$ kaže v smeri rumene puščice [2].

1.2 Ideja za uporabo nevronske mreže

Obravnavajmo enostaven model nematskega tekočega kristala med dvema neskončnima ploščama z dvodimenzionalnim direktorskim poljem

$$\mathbf{n}(z) = (n_x, 0, n_z) = (\cos \theta(z), 0, \sin \theta(z)), \quad (6)$$

kjer kot θ merimo od osi x proti osi z , skladno s sliko 3. Z izbiro sidranja fiksiramo vrednosti $\theta(0)$ in $\theta(d)$. Izraz za gostoto elastične proste energije (1) se v tem primeru poenostavi do

$$f_{elastic} = \frac{1}{2} K_{11} \left(\frac{\partial n_z}{\partial z} \right)^2 + \frac{1}{2} K_{33} \left(\frac{\partial n_x}{\partial z} \right)^2, \quad (7)$$

kjer zvojni člen ne nastopa več, saj v izbrani geometriji (6) vselej velja $\mathbf{n} \cdot \nabla \times \mathbf{n} = 0$. Zanimala nas bo relaksacija direktorskega polja iz nekega začetnega profila $\theta(z, t = 0)$ v ravnovesno stanje $\theta(z, t = \infty)$. To določimo z reševanjem Euler-Lagrangeve enačbe

$$\frac{d}{dz} \left[\frac{\partial f}{\partial \theta'} \right] - \frac{\partial f}{\partial \theta} = \gamma_1 \frac{\partial \theta}{\partial t}, \quad (8)$$

kjer $\theta' = \partial \theta / \partial z$ in člen na desni strani enačbe predstavlja viskozno disipacijo. Konstanto γ_1 v eksperimentu poznamo. Tako dobimo difuzijsko enačbo za relaksacijo $\theta(z, t)$,

$$\frac{\partial^2 \theta}{\partial z^2} (K_{11} \cos^2 \theta + K_{33} \sin^2 \theta) + \frac{1}{2} (K_{33} - K_{11}) \left(\frac{\partial \theta}{\partial z} \right)^2 \sin 2\theta = \gamma_1 \frac{\partial \theta}{\partial t}. \quad (9)$$

Funkcija $\theta(z)$ eksperimentalno ni direktno dostopna, izmerimo pa lahko intenziteto prepuščene svetlobe skozi vzorec. Časovno spreminjanje intenzitete nam tako da podatek o relaksaciji direktorskega polja, ta pa je seveda pogojena z elastičnimi konstantami K_{11} in K_{33} , ki jih želimo določiti. Opišimo torej meritev intenzitete nekoliko podrobneje. Pred celico s tekočim kristalom postavimo polarizator, tako da v celico v smeri osi z vstopi svetlobni žarek, linearno polariziran pod kotom 45° glede na os x . Za celico postavimo še en polarizator, tokrat orientiran pod kotom -45° glede na x , ter izmerimo intenziteto prepuščene svetlobe. Prehod svetlobe skozi celico opišemo z Jonesovim formalizmom,

$$J = \begin{bmatrix} J_x \\ J_y \end{bmatrix} = \underbrace{\frac{1}{2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}}_{\text{izhodni polarizator}} \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & e^{i\Gamma} \end{bmatrix}}_{\text{prehod celice}} \cdot \underbrace{\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}}_{\text{vhodna polarizacija}}, \quad (10)$$

kjer je Γ akumulirana fazna razlika iz enačbe (5). Od tu enostavno dobimo intenziteto prepuščene svetlobe

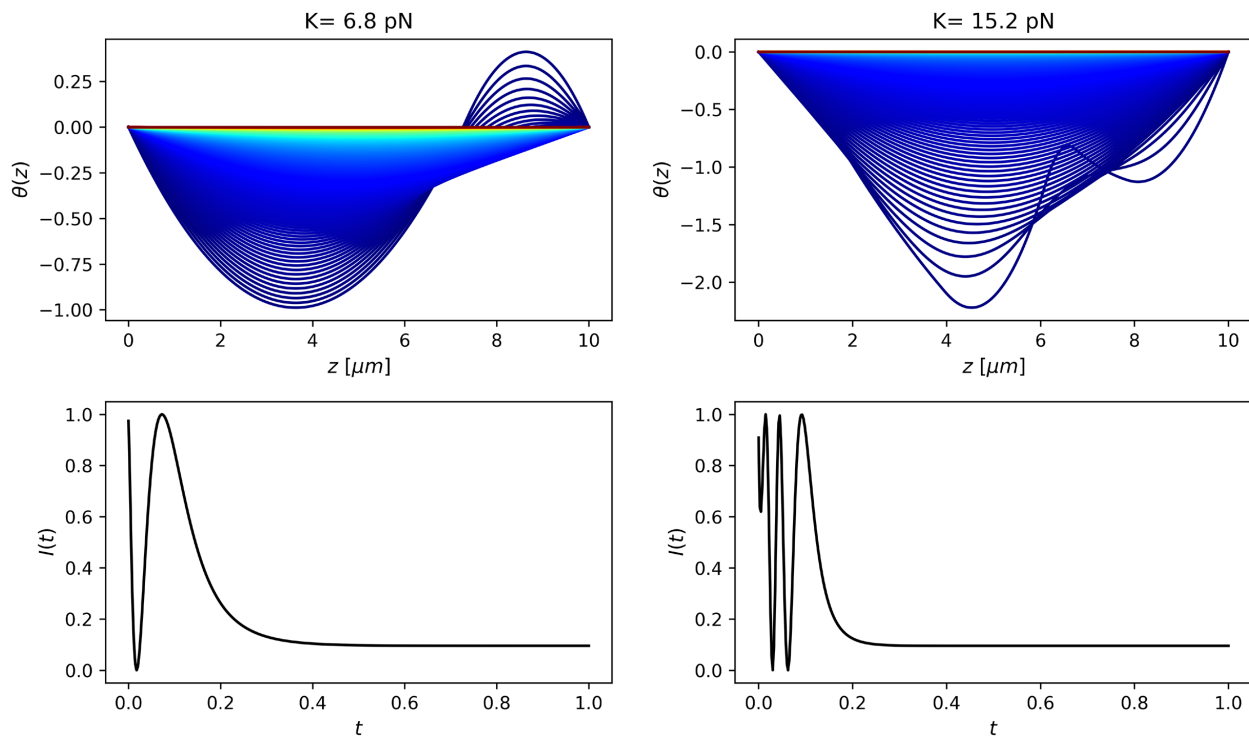
$$I(t) = |J_x|^2 + |J_y|^2 = \sin^2(\Gamma(t)/2), \quad (11)$$

ki se bo tekom relaksacije direktorskega polja spreminjala skladno s spremembo fazne razlike Γ . Cilj naloge je pokazati, da lahko z dobro naučeno nevronske mreže zgolj iz podatka o spreminjanju intenzitete $I(t)$ zanesljivo in natančno določimo elastične konstante nematskega tekočega kristala.

V prvem delu naloge se osredotočimo na primer, ko sta elastični konstanti enaki, $K_{11} = K_{33} = K$, kar poenostavi relaksacijsko enačbo (9) v

$$\frac{K}{\gamma_1} \frac{\partial^2 \theta(z, t)}{\partial z^2} = \frac{\partial \theta(z, t)}{\partial t}. \quad (12)$$

Želimo torej skonstruirati nevronske mreže, ki bi za vsak časovni potek intenzitete $I(t)$ (vhodni podatek mreže $\mathbf{X} = [I_{t=0}, I_{t=\Delta t}, \dots, I_{t=399\Delta t}]$) znala določiti neznano elastično konstanto K (izhodni podatek mreže $\mathbf{Y} = [K]$). Za učenje nevronske mreže potrebujemo veliko število časovnih potekov intenzitete z znanimi K , pridobljenih ob relaksaciji različnih začetnih profilov $\theta(z, t=0)$. Eksperimentalno teh podatkov v zadostnem številu ne moremo pridobiti (potrebovali bi mnogo tekočokristalnih vzorcev z različnimi elastičnimi konstantami), lahko pa jih relativno enostavno generiramo s simulacijo. V ta namen moramo seveda enačbi za relaksacijo direktorskega polja (9) in fazno razliko (5) prepisati v diferenčno obliko primerno za numerično računanje. Simulacija za generiranje posameznega učnega para $(\mathbf{X}, \mathbf{Y})$ v grobem poteka sledeče. Najprej po enakomerni porazdelitvi izžrebamo naključno elastično konstanto z intervala $[0, K_{max} = 20 \text{ pN}]$ in ustvarimo začetni profil $\theta(z, t=0)$. To je naključna gladka funkcija, ki ustreza izbranemu sidranju. Izberemo lahko npr. planarno sidranje $\theta(0, t) = \theta(d, t) = 0$. Ta izbira v poenostavljenem primeru z le eno elastično konstanto namreč ne vpliva na uspešnost učenja nevronske mreže. Pri relaksaciji moramo poskrbeti za dovolj majhen časovni korak (red velikosti 10^{-6}), da bo simulacija numerično stabilna. Ta vrednost je med drugim pogojena tudi z zgornjo mejo konstante K , ki jo želimo določiti, to je K_{max} . Potrebno število korakov (nekajkrat 10^5) je določeno s časom, pri katerem dosežemo ravnovesno stanje – za planarno sidranje bo to $\theta(z) = 0$. Sproti shranjujemo trenutni profil $\theta(z)$ vsakih nekaj (recimo vsakih ~ 500) korakov. Rezultat relaksacije je za dva različna začetna profila $\theta(z, 0)$ prikazan na slikah 4a in b. Za vsakega od N_t shranjenih profilov nato izračunamo intenziteto prepuščene svetlobe in dobimo njen časovni potek $I(t)$ (slika 4c in d).



Slika 4: Časovni razvoj ob relaksaciji funkcije $\theta(t)$ ter pripadajoča časovna poteka prepuščene svetlobe.

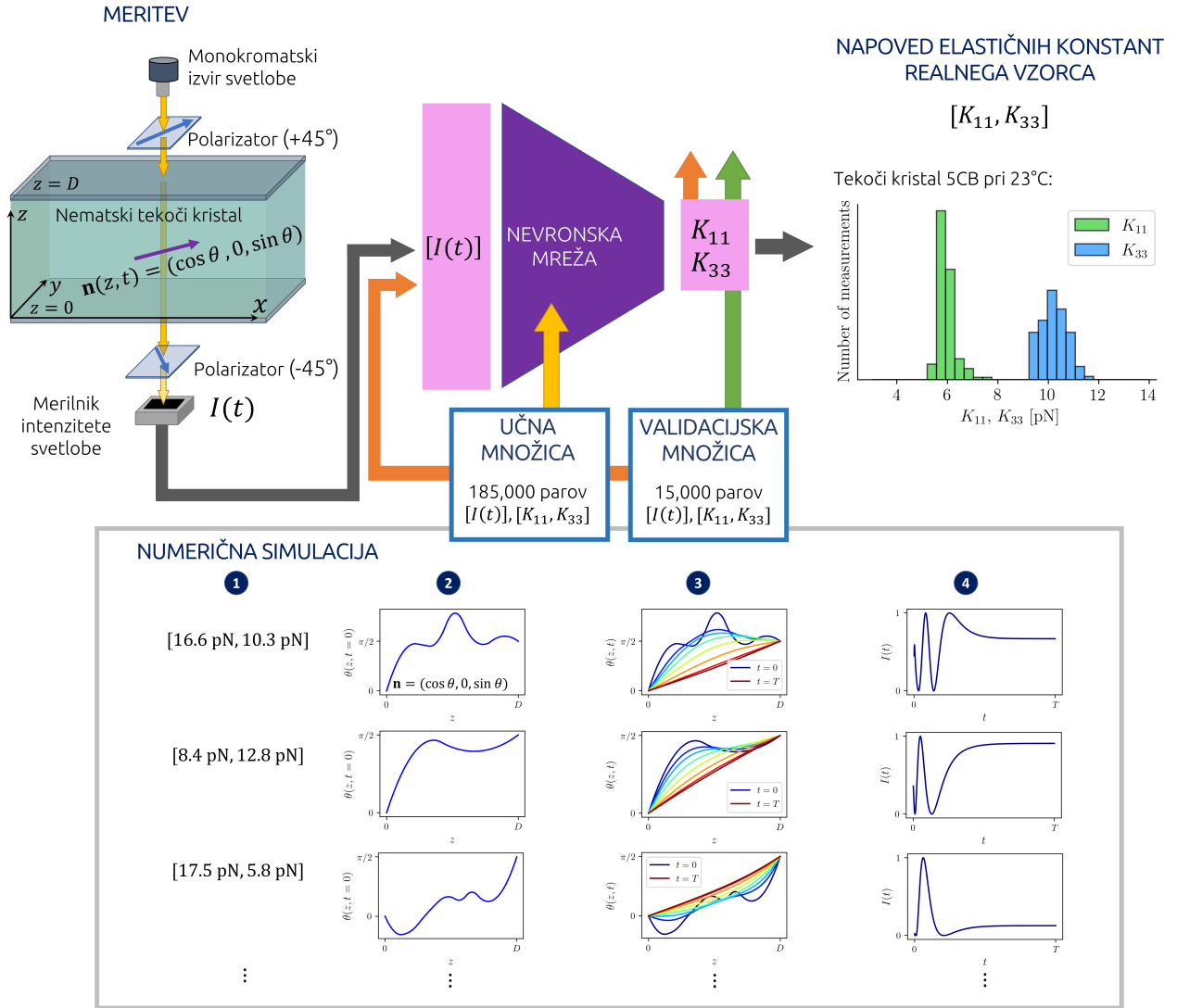
Posamezni časovni potek je torej diskretiziran v N_t točkah in predstavlja vhodni podatek za našo nevronske mrežo, izhodni podatek pa bo konstanta K , ki je v postopku treniranja mreže seveda znana za vsako krivuljo intenzitete. Za učinkovitejše učenje jo vedno normaliziramo $K \rightarrow K/K_{max}$.

Kot bomo ugotovili v drugem delu naloge, lahko z nevronske mrežo določimo obe elastični konstanti iz enačbe (9), K_{11} in K_{33} . Naš končni cilj bo določanje elastičnih konstant vzorca iz eksperimentalnih meritev. Pri tem bomo uporabili nevronske mrežo, ki bo natrenirana na podatkih iz numeričnih simulacij, saj z eksperimenti ne moremo pridobiti dovolj velikega in raznolikega učnega seta. Če v že naučeno nevronske mrežo vstavimo izmerjene poteke intenzitete, nam bo mreža vrnila vrednosti elastičnih konstant za naš vzorec. Celotna shema uporabe nevronske mreže v naši nalogi je prikazana na sliki 5.

1.3 Generacija vhodnih podatkov

Najprej si pogledjmo poenostavljen primer relaksacije v približku ene elastične konstante K . Vhodni podatki so dostopni na strežniku Marvin (`/data/PSUF_naloge/3-naloga/DataK`) v datotekah `"/Kvalues.npy"` (vrednosti konstant K), `"/theta0.npy"` (začetni profili funkcije $\theta(z)$) ter `"/intensity[λ]noise[noise].npy"` (časovni profili; λ je valovna dolžina v nm, noise pa standardni odklon belega šuma $\times 1000$). Datoteke uvozite s funkcijo `numpy.load()`. Podatki vsebujejo 10^5 intenzitetnih krivulj, diskretiziranih v $N_t = 400$ točkah. Dele naloge lahko rešujete na manjšem podatkovnem setu, glede na časovno zahtevnost posamezne simulacije. Z uporabo večjega seta bodo rezultati seveda nekoliko boljši.

Lahko generirate tudi svoje podatke s pomočjo datoteke `DataGeneration.py`, ki je prav tako



Slika 5: Shema uporabe nevronske mreže za določanje elastičnih konstant K_{11} in K_{33} v vzorcu.

dostopna na strežniku Marvin. Za njeno delovanje boste potrebovali nameščeno knjižnjico za paralelno računanje *joblib* ter knjižnjico za prevajanje *python* kode v strojno kodo, imenovano *numba*. Ti poskrbita, da generacija podatkov ne bo trajala predolgo. Na Marvinu znotraj okolja, do katerega dostopate z ukazom

```
source /data/virtualenvs/virtenv-py310-PSUF/bin/activate
```

ta program deluje brez dodatnih instalacij. Datoteko za generacijo lahko nato kot knjižnico uvozite v *python*:

```
from DataGeneration import GenerateData
obj = GenerateData()
obj.generate(10000)
obj.export()
```

Prikazan primer kode ustvari 10000 različnih krivulj intenzitete dolžine $N_t = 400$, ter jih skupaj s pripadajočimi vrednostmi K ter začetnimi profili $\theta(z, 0)$ shrani v mapo "DataK" (privzeta

lokacija te mape je lokacija datoteke *DataGeneration.py*). V metodi `GenerateData.export()` imamo z nastavitvijo parametra `add_intensity_noise` možnost, da (iste) krivulje izvozimo še z dodatkom belega šuma.

2 Nevronska mreža

Za postavitev nevronske mreže si pomagamo s programskim vmesnikom *Keras*, ki je za *Python* na voljo znotraj knjižnice *Tensorflow*.

2.1 Konstrukcija goste mreže

Sestavili bomo "navadno", sekvenčno polno-povezano nevronska mrežo (razred `Sequential`), kjer so med sabo povezani le nevroni iz zaporednih plasti. Ustvarjenemu modelu dodamo plasti z metodo `add()`, kjer kot argument navedemo objekt plasti zelenega tipa. V našem primeru boso vse plasti goste (`Dense`), kar pomeni, da so med sabo z matriko uteži povezani vsi nevroni iz dveh zaporednih plasti, kot argument pa posredujemo število nevronov v plasti ter aktivacijsko funkcijo. Če želimo specificirati inicializacijo uteži ter biasov, to storimo z opcijama `kernel_initializer` ter `bias_initializer`. Pri prvi plasti, ki jo dodamo nevronske mreži, moramo specificirati še dolžino vhodnega vektorja (=vhodna plast), nevroni v zadnji plasti pa služijo kot output mreže. V našem primeru, ko napovedujemo skalar, bo v zadnji plasti torej en sam nevron. Izbira aktivacijske funkcije v zadnji plasti je odvisna tudi od območja, znotraj katerega so definirani izhodni podatki. Ker v našem primeru operiramo z normaliziranimi podatki, je smiselna izbira aktivacijske funkcije z zalogo vrednosti med 0 in 1, npr. `sigmoid`. Lahko bi uporabili tudi funkcijo `relu`, saj fizikalnega razloga za saturacijo v resnici ni, K je lahko večji od izbranega K_{max} . Na modelu nato uporabimo metodo `compile`, kjer izberemo še vrsto optimizatorja ter loss funkcije. V tem koraku lahko dodamo tudi dodatne mere uspešnosti, ki nas zanimajo, z opcijo `metrics`. Inicializacija mreže z vhodno (400 nevronov), dvema skritima (po 200 in 100 nevronov) in izhodno plastjo (1 nevron) izgleda takole:

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(200, activation='sigmoid', input_shape=(400,)))
model.add(Dense(100, activation='sigmoid'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='mean_absolute_error',
              metrics=['mean_squared_error'])
```

Učenje mreže omogoči metoda `fit()`, kamor kot argument vstavimo vhodne in izhodne podatke ter določimo velikost batch-ev ter število epoh. Posebej lahko vstavimo še validacijske podatke (`validation_data`), lahko pa razdelitev prepustimo kar sami metodi – ob uporabi opcije `validation_split` bo namreč izbran delež vzorcev ohranjen za validacijo po vsaki epohi. Metoda `fit()` vrne `History` objekt, kjer so zbrane vrednosti loss funkcije (ter potencialnih drugih mer uspešnosti) za učne podatke in validacijske podatke po vsaki epohi.

Če želimo ovrednotiti uspešnost modela na nekem setu podatkov, uporabimo metodo `evaluate()`. Ko smo model natrenirali, lahko z metodo `predict()`, ki ji posredujemo vhodne

podatke, nazaj dobimo output mreže, torej napoved neznanih konstant K . Prav tako lahko dostopamo do matrik uteži ter biasov za i -to plast (`layers[i].get_weights()`), kar lahko izkoristimo za vizualizacijo mreže.

2.2 Navzkrižna validacija in mrežno iskanje

Če želimo ovrednotiti uspešnost učenja nevronske mreže, pogosto uporabimo k -fold navzkrižno validacijo. Podatke premešamo in jih naključno razdelimo na k delov. Nato k -krat izvedemo učenje (`fit()`) modela, pri čemer en del uporabimo kot testni set podatkov, ostale podatke pa uporabimo za učenje. Po vsakem koraku obdržimo mero uspešnosti modela na testnem setu, sam model pa zavržemo in v naslednjem koraku z učenjem začnemo znova (za testni set seveda zadržimo naslednji del podatkov). Po k ponovitvah učenja lahko uspešnost učenja modela ovrednotimo glede na vse dobljene rezultate: izračunamo lahko povprečno vrednost in varianco dobljenih rezultatov.

Te podatke lahko izkoristimo na dva načina. Za nadaljnjo uporabo lahko izberemo uteži iz tistega fita, ki je vodil do najboljšega rezultata na testnih podatkih. S tem se tudi izognemo možnosti nesrečne razporeditve učnih podatkov, kjer potencialno ne bi zajeli vseh lastnosti celotnega seta podatkov, kar je možno predvsem pri omejenem številu podatkov in za redko zastopane lastnosti. Tak model ne bi imel želene sposobnosti generalizacije na še nikoli "videne" podatke. V splošnem sicer želimo, da je vsak od k setov reprezentativen vzorec vseh podatkov, kar se odraža v majhni varianci rezultatov. Druga možnost uprabe k -fold navzkrižne validacije nastopi za primerjavo uspešnosti med modeli z različnimi hiperparametri. Zato se pogosto uporablja kot temelj mrežnega iskanja (grid search) optimalnih hiperparametrov modela. Ti so specifični za vsak problem in v splošnem ne moremo sklepati, da bodo podobni kot pri nekem drugem, četudi sorodnem problemu.

Za implementacijo navzkrižne validacije ter mrežnega iskanja si pomagamo s knjižnico *sklearn*. k -fold validacijo izvedemo s pomočjo razreda `Kfold`, kjer kot parameter `n_splits` navedemo želeno število razdelitev podatkov. Nato ustvarimo zanko čez vse učne ter testne (validacijske) sete, ki jih proizvede generator `split`, uporabljen na prej ustvarjeni instanci `Kfold`. Znotraj zanke vsakič znova ustvarimo nov model (nevronske mreže), jo natreniramo na učnih podatkih ter iz vrednotimo mere uspešnosti na validacijskih podatkih, ki jih shranjujemo v nek seznam, imamo torej

```
scores_per_fold = []
for train, test in kf.split(X, Y):
    model = create_model()
    history = model.fit(X[train], Y[train],
                        epochs=100, batch_size=100, verbose=0)

    scores = model.evaluate(X[test], Y[test], verbose=0)
    scores_per_fold.append(scores)
```

kjer s funkcijo `create_model` ustvarimo model kot je prikazano zgoraj (vključno z metodo `compile`). Seveda lahko znotraj zanke shranjujemo tudi kakšne druge parametre. Če mere uspešnosti za validacijske podatke iz vrednotimo po vsaki epohi (dobili bi torej graf spreminjanja tekom učenja), lahko po koncu navzkrižne validacije za posamezni model izrišemo povprečni graf spreminjanja mer uspešnosti za ta model. S tem zmanjšamo zašumljenost grafov in olajšamo primerjavo med modeli.

Za izvedbo mrežnega iskanja se bomo močnejše oprli na knjižnico *sklearn*. Naš model nevronske mreže, ustvarjen s *Kerasom*, zavijemo v razred **KerasRegressor**, kjer kot argument **build_fn** navedemo funkcijo, ki vrača model. Veljavni argumenti so tudi vsi, ki jih lahko posredujemo metodi **fit** za naš model, npr. število batchev ter epoh, ter potencialni argumenti funkcije, posredovane v **build_fn**. Izbrane vrednosti hiperparametrov, za katere želimo izvesti mrežno iskanje, shranimo v slovar, kjer kot ključe navedemo imena spremenljivk. Ta se morajo ujemati s spremenljivkami, ki jih lahko sprejme metoda **fit** ali naša generacijska funkcija modela. Nato lahko ustvarimo instanco razreda **GridSearchCV**, kjer naš model posredujemo kot **estimator** ter slovar hiperparametrov kot **param_grid**. Z opcijo **cv** lahko nastavimo število razdelitev za navzkrižno validacijo, posebej pa lahko specificiramo tudi kriterij, po katerem bomo ocenjevali uspešnost posameznih modelov, z opcijo **scoring**. **n_jobs** nam pove število jeder procesorja, ki jih uporabimo za paralelno računanje, vrednost -1 pomeni vsa razpoložljiva jedra. Nato lahko uporabimo metodo **fit** na instanci **GridSearchCV**, rezultati pa so dostopni pod atributom **cv_results_** (slovar). Okvirno tako za določitev najustreznejšega optimizatorja ter aktivacijske funkcije dobimo

```
model = KerasRegressor(build_fn=create_model, epochs=100,
                        batch_size=50, verbose=0)

optimizer = ['SGD', 'RMSprop', 'Adam']
activation = ['sigmoid', 'linear', 'relu', 'tanh']
param_grid = dict(activation=activation, optimizer=optimizer)

grid = GridSearchCV(estimator=model, param_grid=param_grid,
                    n_jobs=-1, cv=3)
grid.fit(X, Y)

means = result.cv_results_['mean_test_score']
stds = result.cv_results_['std_test_score']
params = result.cv_results_['params']
```

Pri tem seveda pazimo, da funkcija **create_model**, ki ustvari naš model, sprejema parametre z imeni **optimizer** in **activation**.

3 Naloga in vprašanja

PRVI DEL

1. Uvozite ali generirajte podatke in se spoznajte s podatkovnim setom. Preverite, katerim vrednostim konstante K ter začetnim profilom pripadajo krivulje intenzitete, ki odstopajo od najbolj tipičnih. Za določitev časovnih potekov $\theta(z, t)$ iz nekega začetnega profila lahko uporabite funkcijo **theta.time.evolution** iz *DataGeneration.py*. Upoštevajte $\gamma_1 = 0.098$.
2. Postavite nevronske mreže ter jo natrenirajte na generiranih podatkih. Preverite, kako poteka učenje za različne hiperparametre, npr. loss funkcije, aktivacijske funkcije, število plasti ..., spremljate pa lahko tudi časovno zahtevnost za posamezne izbire. Uporabite k -fold navzkrižno validacijo za lepše rezultate z manj šuma.

3. Najbolj pomembna je učinkovitost modela na še ne videnih podatkih. Narišite graf, na katerem primerjajte prave in napovedane vrednosti konstante K za večje število podatkov. Izgleda, da pride do sistematičnega odstopanja za del intenzitetnih krivulj – lahko ugotovimo, katere so to?
4. Z navzkrižno validacijo ter mrežnim iskanjem poskusite določiti optimalne hiperparametre za naš primer, npr. število batchev in epoh, aktivacijske funkcije... Lahko poskusite poiskati optimalne parametre navadnega SGD optimizatorja (learning rate, momentum, nastavite learning rate decay iz `keras.optimizers.schedules`) in pogledate, če se uspete približati učinkovitosti optimizatorja "adam".
5. Kaj opazite, če nevronske mreže učite na podatkih, ki imajo dodan beli šum? Kaj pa če jo učite na podatkih brez šuma in jo potem uporabite za napovedovanje iz podatkov, ki vsebujejo šum?
6. Če ne spreminjate parametrov v generatorju podatkov, je $I(t)$ diskretizirana v 400 časovnih točkah na intervalu $[0\text{ s}, T = 1.2\text{ s}]$. Kako na rezultate vpliva, če intenziteto na istem časovnem intervalu interpolirate v le 200, 100, 50 ali še manj časovnih točkah? Tedaj je seveda tudi vhodna plast nevronske mreže drugačna. Kaj pa če učite mrežo z $I(t)$ podanem le na enem izseku izvirnega intervala, npr. na $[T/10 = 0.12\text{ s}, T = 1.2\text{ s}]$ ali kakšnem drugem?

DRUGI DEL

7. V mapah "/DataK13*" se nahajajo simulacijski rezultati za 1.2 sekund trajajočo relaksacijo z obema elastičnima konstantama, K_{11} in K_{33} , pri različnih kombinacijah sidranja (planarno-planarno, planarno-homeotropno, homeotropno-homeotropno). Sestavite nevronske mreže za prepoznavanje obeh konstant ter preverite, pri kakšnih robnih pogojih lahko dobro napovemo posamezno konstanto. Zakaj je napoved v določenih primerih slaba?
8. V mapi "/ExpData" lahko najdete 30 izmerjenih intenzitetnih krivulj za vzorec tekočega kristala 5CB na časovnem intervalu $[0\text{ s}, 1.95\text{ s}]$ v 15600 časovnih točkah:

`(time=np.linspace(0,1.95,15600)).`

Uporabljena je bila celica debeline $10\mu\text{m}$, kombinacija planarnega in homeotropnega sidranja ter svetilo z valovno dolžino $\lambda = 505\text{ nm}$. Pri sobni temperaturi je rotacijska viskoznost za 5CB $\gamma_1 = 0.098\text{ Pa s}$, lomna količnika pa sta $n_o = 1.545$, $n_e = 1.740$, kar je že upoštevano v generiranih numeričnih podatkih.

Če niste spreminjali parametrov v generatorju podatkov ali če uporabljate že generirane podatke, potem nevronska mreža za vhodni podatek pozna izmerjeno intenziteto $I(t)$, ki je diskretizirana v 400 časovnih točkah na intervalu $[0\text{ s}, 1.2\text{ s}]$. Da eksperimentalne meritve preoblikujete v format, ki je kompatibilen z nevronske mreže, si lahko pomagata z interpolacijo (funkcija `interp1d` iz knjižnice *scipy*). Poskusite z generiranimi podatki naučiti nevronske mreže tako, da bodo napovedi na eksperimentalnih podatkih čim boljše. Kako izbira hiperparametrov vpliva na porazdelitev napovedi elastičnih konstant vzorca? V rešnici sta pri sobni temperaturi elastični konstanti $K_{11} \approx 6.6\text{ pN}$ in $K_{33} \approx 9.0\text{ pN}$ [7]. Eksperimentalne rezultate je pripravil izr. prof. dr. Miha Škarabot iz IJS, Odsek F5.

9. Za izboljšanje rezultatov lahko poskusite pridobiti "več meritev" s tem, da iz posamezne izmerjene intenzitetne krivulje vzamete več intervalov z različnimi začetnimi časi. Premislite, zakaj je tak pristop smiseln. Kako se pri tem spremeni napoved elastičnih konstant?
10. Inicializacija uteži posamezne nevronske mreže je pred vsakim učenjem naključna, zato tudi mreže z identično arhitekturo in z uporabo istih učnih hiperparametrov po koncu učenja delujejo nekoliko drugačne. Ali opazite izboljšanje napovedi, če elastične konstante napoveste z ansamblom (petih, desetih ali še več) nevronskih mrež in rezultate potem povprečite?

4 Dodatek: optimizatorji

Običajen gradientni spust ni najbolj učinkovit optimizator. Njegovo hitrost zadržuje predvsem fiksna vrednost hitrosti učenja, kar lahko povzroči, da do minimuma loss funkcije pridemo zelo počasi ali pa ga (pri prevelikem koraku) celo zgrešimo. Različne tehnike izboljšanja gradientnega spusta temeljijo predvsem na dinamični izbiri hitrosti učenja. Spodaj je na kratko opisanih nekaj naprednejših algoritmov.

- "adagrad" – za izračun spremembe uteži θ_i velikost koraka delimo z povprečno vrednostjo kvadratov (varianco) vseh prejšnjih gradientov,

$$\Delta\theta_i = -\eta \frac{g_i^{(t)}}{\sqrt{\sum_{t' \leq t} (g_i^{(t')})^2}}, \quad (13)$$

kjer je $g_i^{(t)} = \partial J / \partial \theta_i$ velikost gradienta loss funkcije J ob času (koraku) t . Tako preprečimo, da ob majhnih vrednostih gradienta delamo zelo majhne korake, vendar bo vsota v imenovalcu s časom naraščala, kar lahko upočasni učenje pri večjih časih.

- "RMSprop" – če te upočasnitve učenja s časom ne želimo, lahko pri skaliranju koraka vzamemo le podatke o gradientih za zadnjih nekaj korakov (drseče povprečje). To lahko modeliramo z "razpadno konstanto" $\gamma_1 < 1$, ki pri posodobitvi ocene variance ob času t , $V^{(t)}$, pove, v kolikšni meri ohranimo vrednost ob času $t - 1$ in kolikšen je vpliv trenutno izračunanega gradienta,

$$V^{(t)} = \gamma V^{(t-1)} + (1 - \gamma)(g_i^{(t)})^2 \rightarrow V^{(t)} = (1 - \gamma) \sum_{t' \leq t} \gamma^{t-t'} (g_i^{(t')})^2. \quad (14)$$

Izračunani gradienti bodo tako v oceni variance s časom potenčno izgubljali vpliv na vrednost $V^{(t)}$. Za posodobitev uteži dobimo $\Delta\theta_i = -\eta g_i^{(t)} / \sqrt{V^{(t)}}$.

- "adam" – na voljo šele od leta 2014. Tu ne gledamo le drsečega povprečja variance, temveč tudi drseče povprečje gradienta,

$$m^{(t)} = \beta m^{(t-1)} + (1 - \beta) g^{(t)}, \quad (15)$$

kjer je $m^{(t)}$ ocena za gradient. Ocena variance je izračunana enako kot v (14). Te vrednosti nato še popravimo kot

$$\hat{m}^{(t)} = \frac{m^{(t)}}{1 - \beta^t}, \quad (16)$$

$$\hat{V}^{(t)} = \frac{V^{(t)}}{1 - \gamma^t}, \quad (17)$$

s čimer odpravimo napako zaradi inicializacije $m^{(0)} = 0$ in $V^{(0)} = 0$. Uteži nato posodobimo kot $\Delta\theta_i = -\eta \frac{\hat{m}^{(t)}}{\sqrt{\hat{V}^{(t)}}}$.

Literatura

- [1] Klus, B., Laudyn, U. A., Karpierz, M. A., Sahraoui, B., *All-optical measurement of elastic constants in nematic liquid crystals*, Opt. Express, **22**, 30257 (2014).
- [2] Huang, Y., Liao, E., Chen, R., Wu, S. *Liquid-Crystal-on-Silicon for Augmented Reality Displays*, Appl. Sci. **8**, 2366, (2018).
- [3] Kingma, D. P., Ba, J., *Adam: a method for stochastic optimization*, arXiv:1412.6980 [cs.LG] (2014).
- [4] *How to use K-fold Cross Validation with Keras?*, <https://www.machinecurve.com/index.php/2020/02/18/how-to-use-k-fold-cross-validation-with-keras/>.
- [5] Brownlee, J., *How to Grid Search Hyperparameters for Deep Learning Models in Python With Keras*, <https://machinelearningmastery.com/grid-search-hyperparameters-deep-learning-models-python-keras/>.
- [6] Marquardt, F., *Online Course: Machine learning for physicists 2020*, https://pad.gwdg.de/s/HJtiTE__U.
- [7] M. Cui in J. Kelly, *Temperature dependence of visco-elastic properties of 5CB*, Informa UK Publishing **331**, 49 (1999).